

P1831R1

Deprecating **volatile**: library

Published Proposal, 2020-02-12

This version:

<http://wg21.link/P1831R1>

Author:

[JF Bastien](#) (Apple)

Audience:

LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/jfbastien/papers/blob/master/source/P1831R1.bs

Table of Contents

1 Abstract

2 Edit History

2.1 r0 → r1

3 Wording

3.1 Tuples [**tuple**]

3.2 Variants [**variant**]

3.3 Atomic operations library [**atomics**]

3.4 Annex D

3.4.1 Tuple [**depr.tuple**]

3.4.2 Variant [**depr.variant**]

3.4.3 Atomic operations library [**depr.atomics**]

References

§ 1. Abstract

This paper is the library part of P1152. The Core language parts of the deprecation were voted into C++20 at the Cologne meeting as [\[P1152R4\]](#). LWG was unable to review library wording, this paper therefore carries forward the library parts of [\[P1152R3\]](#).

§ 2. Edit History

§ 2.1. r0 → r1

Address LWG feedback:

- Change one "they" to "it".
- Don't use *Mandates:* clause, use *Constraints:* instead.
- Clarify why we don't have a feature test macro.

§ 3. Wording

The proposed wording follows the library approach to deprecation: library deprecation presents the library without the deprecated feature, and only mentions said feature in Annex D.

No feature test macro is added, per SG10 guidance: developers cannot use a feature test macro to decide to do something else after this deprecation, they should instead fix the code even before the deprecation.

§ 3.1. Tuples **[tuple]**

Modify as follows.

Header <tuple> synopsis [**tuple.syn**]:

```
namespace std {

[...]
```

// [*tuple.helper*], tuple helper classes

```
template<class T> class tuple_size;           // not defined
template<class T> class tuple_size<const T>;
template<class T> class tuple_size<volatile T>;
template<class T> class tuple_size<const volatile T>;

template<class... Types> class tuple_size<tuple<Types...>>;

template<size_t I, class T> class tuple_element;    // not defined
template<size_t I, class T> class tuple_element<I, const T>;
template<size_t I, class T> class tuple_element<I, volatile T>;
template<size_t I, class T> class tuple_element<I, const volatile T>;

[...]
```

```
}
```

[...]

Tuple helper classes [**tuple.helper**]

```
template<class T> class tuple_size<const T>;
template<class T> class tuple_size<volatile T>;
template<class T> class tuple_size<const volatile T>;
```

Let TS denote `tuple_size<T>` of the cv-unqualified type T. If the expression `TS::value` is well-formed when treated as an unevaluated operand, then ~~each of the three templates~~ **the template** shall satisfy the TransformationTrait requirements with a base characteristic of

```
integral_constant<size_t, TS::value>
```

Otherwise, ~~they~~ **it** shall have no member value.

Access checking is performed as if in a context unrelated to TS and T. Only the validity of the immediate context of the expression is considered. [*Note*: The compilation of the expression can

result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the "immediate context" and can result in the program being ill-formed. —*end note*]

In addition to being available via inclusion of the `<tuple>` header, the ~~three templates are~~ template is available when any of the headers `<array>`, `<ranges>`, or `<utility>` are included.

```
template<size_t I, class T> class tuple_element<I, const T>;  
template<size_t I, class T> class tuple_element<I, volatile T>;  
template<size_t I, class T> class tuple_element<I, const volatile T>;
```

Let TE denote `tuple_element_t<I, T>` of the cv-unqualified type T. Then ~~each of the three templates~~ the template shall satisfy the TransformationTrait requirements with a member typedef type that names the ~~following type :~~ add_const_t<TE>.

- ~~• for the first specialization, add_const_t<TE>;~~
- ~~• for the second specialization, add_volatile_t<TE>; and~~
- ~~• for the third specialization, add_cv_t<TE>;~~

In addition to being available via inclusion of the `<tuple>` header, the ~~three templates are~~ template is available when any of the headers `<array>`, `<ranges>`, or `<utility>` are included.

§ 3.2. Variants [variant]

Modify as follows.

<variant> synopsis [**variant.syn**]

```
namespace std {
// [variant.variant], class template variant
template<class... Types>
    class variant;

// [variant.helper], variant helper classes
template<class T> struct variant_size; // not defined
template<class T> struct variant_size<const T>;
template<class T> struct variant_size<volatile T>;
template<class T> struct variant_size<const volatile T>;
template<class T>
    inline constexpr size_t variant_size_v = variant_size<T>::value;

template<class... Types>
    struct variant_size<variant<Types...>>;

template<size_t I, class T> struct variant_alternative; // not defined
template<size_t I, class T> struct variant_alternative<I, const T>;
template<size_t I, class T> struct variant_alternative<I, volatile T>;
template<size_t I, class T> struct variant_alternative<I, const volatile T>;

[...]
```



variant helper classes [**variant.helper**]

```
template<class T> struct variant_size;
```

Remark: All specializations of `variant_size` shall satisfy the `UnaryTypeTrait` requirements with a base characteristic of `integral_constant<size_t, N>` for some `N`.

```
template<class T> class variant_size<const T>;
template<class T> class variant_size<volatile T>;
template<class T> class variant_size<const volatile T>;
```

Let VS denote `variant_size<T>` of the cv-unqualified type T. Then ~~each of the three templates~~ the template shall satisfy the `UnaryTypeTrait` requirements with a base characteristic of `integral_constant<size_t, VS::value>`.

```
template<class... Types>
    struct variant_size<variant<Types...>> : integral_constant<size_t, siz
```

```
template<size_t I, class T> class variant_alternative<I, const T>;
template<size_t I, class T> class variant_alternative<I, volatile T>;
template<size_t I, class T> class variant_alternative<I, const volatile
```

Let VA denote `variant_alternative<I, T>` of the cv-unqualified type T. Then ~~each of the three templates~~ the template shall meet the `TransformationTrait` requirements with a member typedef type that names the ~~following~~ type : `add_const_t<VA::type>`.

- ~~for the first specialization, `add_const_t<VA::type>`;~~
- ~~for the second specialization, `add_volatile_t<VA::type>`, and~~
- ~~for the third specialization, `add_cv_t<VA::type>`.~~

§ 3.3. Atomic operations library [**atomics**]

Modify as follows.

Operations on atomic types [**atomics.types.operations**]

[*Note*: Many operations are `volatile`-qualified. The "volatile as device register" semantics have not changed in the standard. This qualification means that volatility is preserved when applying these operations to volatile objects. It does not mean that operations on non-volatile objects become volatile. —*end note*]

[...]

```
bool is_lock_free() const volatile noexcept;  
bool is_lock_free() const noexcept;
```

Returns: `true` if the object's operations are lock-free, `false` otherwise.

[*Note*: The return value of the `is_lock_free` member function is consistent with the value of `is_always_lock_free` for the same type. —*end note*]

```
void store(T desired, memory_order order = memory_order::seq_cst) volatile  
void store(T desired, memory_order order = memory_order::seq_cst) noexcept
```

Requires: The order argument shall not be `memory_order::consume`, `memory_order::acquire`, nor `memory_order::acq_rel`.

Constraints: For the `volatile` overload of this function, `atomic<T>::is_always_lock_free` is `true`.

Effects: Atomically replaces the value pointed to by `this` with the value of `desired`. Memory is affected according to the value of `order`.

```
T operator=(T desired) volatile noexcept;  
T operator=(T desired) noexcept;
```

Constraints: For the `volatile` overload of this function, `atomic<T>::is_always_lock_free` is `true`.

Effects: Equivalent to `store(desired)`.

Returns: `desired`.

```
T load(memory_order order = memory_order::seq_cst) const volatile noexcept  
T load(memory_order order = memory_order::seq_cst) const noexcept;
```

Requires: The order argument shall not be `memory_order::release` nor `memory_order::acq_rel`.

Constraints: For the volatile overload of this function, `atomic<T>::is_lock_free` is true.

Effects: Memory is affected according to the value of order.

Returns: Atomically returns the value pointed to by `this`.

```
operator T() const volatile noexcept;  
operator T() const noexcept;
```

Constraints: For the volatile overload of this function, `atomic<T>::is_lock_free` is true.

Effects: Equivalent to: `return load();`

```
T exchange(T desired, memory_order order = memory_order::seq_cst) volatile  
T exchange(T desired, memory_order order = memory_order::seq_cst) noexcept
```

Constraints: For the volatile overload of this function, `atomic<T>::is_lock_free` is true.

Effects: Atomically replaces the value pointed to by `this` with `desired`. Memory is affected according to the value of order. These operations are atomic read-modify-write operations.

Returns: Atomically returns the value pointed to by `this` immediately before the effects.

```

bool compare_exchange_weak(T& expected, T desired,
                           memory_order success, memory_order failure) v
bool compare_exchange_weak(T& expected, T desired,
                           memory_order success, memory_order failure) n
bool compare_exchange_strong(T& expected, T desired,
                             memory_order success, memory_order failure)
bool compare_exchange_strong(T& expected, T desired,
                             memory_order success, memory_order failure)
bool compare_exchange_weak(T& expected, T desired,
                           memory_order order = memory_order::seq_cst) v
bool compare_exchange_weak(T& expected, T desired,
                           memory_order order = memory_order::seq_cst) n
bool compare_exchange_strong(T& expected, T desired,
                             memory_order order = memory_order::seq_cst)
bool compare_exchange_strong(T& expected, T desired,
                             memory_order order = memory_order::seq_cst)

```

Requires: The failure argument shall not be `memory_order::release` nor `memory_order::acq_rel`.

Constraints: For the volatile overload of this function, `atomic<T>::is_always_lock_free` is true.

Effects: Retrieves the value in `expected`. It then atomically compares the value representation of the value pointed to by `this` for equality with that previously retrieved from `expected`, and if true, replaces the value pointed to by `this` with that in `desired`. If and only if the comparison is true, memory is affected according to the value of `success`, and if the comparison is false, memory is affected according to the value of `failure`. When only one `memory_order` argument is supplied, the value of `success` is `order`, and the value of `failure` is `order` except that a value of `memory_order::acq_rel` shall be replaced by the value `memory_order::acquire` and a value of `memory_order::release` shall be replaced by the value `memory_order::relaxed`. If and only if the comparison is false then, after the atomic operation, the value in `expected` is replaced by the value pointed to by `this` during the atomic comparison. If the operation returns `true`, these operations are atomic read-modify-write operations on the memory pointed to by `this`. Otherwise, these operations are atomic load operations on that memory.

Returns: The result of the comparison.

[...]

Specializations for integers [**atomics.types.int**]

```
T fetch_key(T operand, memory_order order = memory_order::seq_cst) volatile  
T fetch_key(T operand, memory_order order = memory_order::seq_cst) noexcept
```

Constraints: For the volatile overload of this function, `atomic<T>::is_always_lock_free` is true.

Effects: Atomically replaces the value pointed to by `this` with the result of the computation applied to the value pointed to by `this` and the given operand. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations.

Returns: Atomically, the value pointed to by `this` immediately before the effects.

Remarks: For signed integer types, the result is as if the object value and parameters were converted to their corresponding unsigned types, the computation performed on those types, and the result converted back to the signed type. [*Note:* There are no undefined results arising from the computation. —end note]

```
T operator op=(T operand) volatile noexcept;  
T operator op=(T operand) noexcept;
```

Constraints: For the volatile overload of this function, `atomic<T>::is_always_lock_free` is true.

Effects: Equivalent to: `return fetch_key(operand) op operand;`

Specializations for floating-point types [**atomics.types.float**]

The following operations perform arithmetic addition and subtraction computations. The key, operator, and computation correspondence are identified in [**atomic.arithmetic.computations**].

```
T A::fetch_key(T operand, memory_order order = memory_order_seq_cst) volatile  
T A::fetch_key(T operand, memory_order order = memory_order_seq_cst) noexcept
```

Constraints: For the volatile overload of this function, `atomic<T>::is_always_lock_free` is true.

Effects: Atomically replaces the value pointed to by `this` with the result of the computation applied to the value pointed to by `this` and the given operand. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations.

Returns: Atomically, the value pointed to by `this` immediately before the effects.

Remarks: If the result is not a representable value for its type the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on *floating-point* should conform to the `std::numeric_limits<floating-point>` traits associated with the floating-point type. The floating-point environment for atomic arithmetic operations on *floating-point* may be different than the calling thread's floating-point environment.

```
T operator op=(T operand) volatile noexcept;  
T operator op=(T operand) noexcept;
```

Constraints: For the volatile overload of this function, `atomic<T>::is_always_lock_free` is true.

Effects: Equivalent to: `return fetch_key(operand) op operand;`

Remarks: If the result is not a representable value for its type the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on *floating-point* should conform to the `std::numeric_limits<floating-point>` traits associated with the floating-point type. The floating-point environment for atomic arithmetic operations on *floating-point* may be different than the calling thread's floating-point environment.

Partial specialization for pointers [**atomics.types.pointer**]

```
T* fetch_key(ptrdiff_t operand, memory_order order = memory_order::seq_c  
T* fetch_key(ptrdiff_t operand, memory_order order = memory_order::seq_c  
◀────────────────────────────────────────────────────────────────────────▶
```

Requires: T shall be an object type, otherwise the program is ill-formed. [*Note:* Pointer arithmetic on `void*` or function pointers is ill-formed. —end note]

Constraints: For the volatile overload of this function, `atomic<T>::is_always_lock_free` is true.

Effects: Atomically replaces the value pointed to by `this` with the result of the computation applied to the value pointed to by `this` and the given operand. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations.

Returns: Atomically, the value pointed to by `this` immediately before the effects.

Remarks: The result may be an undefined address, but the operations otherwise have no undefined behavior.

```
T* operator op=(ptrdiff_t operand) volatile noexcept;  
T* operator op=(ptrdiff_t operand) noexcept;
```

Constraints: For the volatile overload of this function, `atomic<T>::is_lock_free` is true.

Effects: Equivalent to: `return fetch_key(operand) op operand;`

Member operators common to integers and pointers to objects [**atomics.memop**]

```
T operator++(int) volatile noexcept;  
T operator++(int) noexcept;
```

Constraints: For the volatile overload of this function, `atomic<T>::is_lock_free` is true.

Effects: Equivalent to: `return fetch_add(1);`

```
T operator--(int) volatile noexcept;  
T operator--(int) noexcept;
```

Constraints: For the volatile overload of this function, `atomic<T>::is_lock_free` is true.

Effects: Equivalent to: `return fetch_sub(1);`

```
T operator++() volatile noexcept;  
T operator++() noexcept;
```

Constraints: For the volatile overload of this function, `atomic<T>::is_lock_free` is true.

Effects: Equivalent to: `return fetch_add(1) + 1;`

```
T operator--() volatile noexcept;  
T operator--() noexcept;
```

Constraints: For the volatile overload of this function, `atomic<T>::is_lock_free` is true.

Effects: Equivalent to: `return fetch_sub(1) - 1;`

Non-member functions [**atomics.nonmembers**]

A non-member function template whose name matches the pattern `atomic_f` or the pattern `atomic_f_explicit` invokes the member function `f`, with the value of the first parameter as the object expression and the values of the remaining parameters (if any) as the arguments of the member function call, in order. An argument for a parameter of type `atomic<T>::value_type*`

is dereferenced when passed to the member function call. If no such member function exists, the program is ill-formed.

```
template<class T>
    void atomic_init(volatile atomic<T>* object, typename atomic<T>::value
template<class T>
    void atomic_init(atomic<T>* object, typename atomic<T>::value_type des
```

Constraints: For the volatile overload of this function, `atomic<T>::is_always_lock_free` is true.

Effects: Non-atomically initializes *object with value desired. This function shall only be applied to objects that have been default constructed, and then only once. [*Note:* These semantics ensure compatibility with C. —end note] [*Note:* Concurrent access from another thread, even via an atomic operation, constitutes a data race. —end note]

[*Note:* The non-member functions enable programmers to write code that can be compiled as either C or C++, for example in a shared header file. —end note]

§ 3.4. Annex D

Add the following wording to Annex D:

§ 3.4.1. Tuple [depr.tuple]

Header `<tuple>` synopsis [depr.tuple.syn]:

```

namespace std {

[...]

// [tuple.helper], tuple helper classes
template<class T> class tuple_size<volatile T>;
template<class T> class tuple_size<const volatile T>;

template<size_t I, class T> class tuple_element<I, volatile T>;
template<size_t I, class T> class tuple_element<I, const volatile T>;

[...]

}

```

Tuple helper classes [**depr.tuple.helper**]

```

template<class T> class tuple_size<volatile T>;
template<class T> class tuple_size<const volatile T>;

```

Let **TS** denote `tuple_size<T>` of the cv-unqualified type **T**. If the expression `TS::value` is well-formed when treated as an unevaluated operand, then each of the two templates shall satisfy the **TransformationTrait** requirements with a base characteristic of

```

integral_constant<size_t, TS::value>

```

Otherwise, they shall have no member `value`.

Access checking is performed as if in a context unrelated to **TS** and **T**. Only the validity of the immediate context of the expression is considered.

In addition to being available via inclusion of the `<tuple>` header, the two templates are available when any of the headers `<array>`, `<ranges>`, or `<utility>` are included.

```
template<size_t I, class T> class tuple_element<I, volatile T>;
template<size_t I, class T> class tuple_element<I, const volatile T>;
```

Let `TE` denote `tuple_element_t<I, T>` of the cv-unqualified type `T`. Then each of the two templates shall satisfy the `TransformationTrait` requirements with a member typedef `type` that names the following type:

- for the first specialization, `add_volatile_t<TE>`, and
- for the second specialization, `add_cv_t<TE>`.

In addition to being available via inclusion of the `<tuple>` header, the two templates are available when any of the headers `<array>`, `<ranges>`, or `<utility>` are included.

§ 3.4.2. Variant [depr.variant]

`<variant>` synopsis [depr.variant.syn]

```
namespace std {

    // [variant.helper], variant helper classes
    template<class T> struct variant_size<volatile T>;
    template<class T> struct variant_size<const volatile T>;

    template<size_t I, class T> struct variant_alternative<I, volatile T>;
    template<size_t I, class T> struct variant_alternative<I, const volatile T>;

}
```

`variant` helper classes [depr.variant.helper]

```
template<class T> class variant_size<volatile T>;
template<class T> class variant_size<const volatile T>;
```

Let `VS` denote `variant_size<T>` of the cv-unqualified type `T`. Then each of the two templates shall satisfy the `UnaryTypeTrait` requirements with a base characteristic of `integral_constant<size_t, VS::value>`.

```
template<size_t I, class T> class variant_alternative<I, volatile T>;
template<size_t I, class T> class variant_alternative<I, const volatile
```

Let `VA` denote `variant_alternative<I, T>` of the cv-unqualified type `T`. Then each of the two templates shall meet the `TransformationTrait` requirements with a member typedef `type` that names the following type:

- for the first specialization, `add_volatile_t<VA::type>`, and
- for the second specialization, `add_cv_t<VA::type>`.

§ 3.4.3. Atomic operations library [depr.atomics]

If an atomic specialization has one of the following overloads, then that overload is available when `atomic<T>::is_always_lock_free` is `false`:

```

void store(T desired, memory_order order = memory_order::seq_cst) volatile;
T operator=(T desired) volatile noexcept;
T load(memory_order order = memory_order::seq_cst) const volatile noexcept;
operator T() const volatile noexcept;
T exchange(T desired, memory_order order = memory_order::seq_cst) volatile;
bool compare_exchange_weak(T& expected, T desired, memory_order success;
bool compare_exchange_strong(T& expected, T desired, memory_order success;
bool compare_exchange_weak(T& expected, T desired, memory_order order =
bool compare_exchange_strong(T& expected, T desired, memory_order order =
T fetch_key(T operand, memory_order order = memory_order::seq_cst) volatile;
T operator op=(T operand) volatile noexcept;
T* fetch_key(ptrdiff_t operand, memory_order order = memory_order::seq_cst) volatile;

```

The following non-member function is available when `atomic<T>::is_always_lock_free` is `false`:

```

template<class T>
void atomic_init(volatile atomic<T>* object, typename atomic<T>::value_type value);

```

§ References

§ Informative References

[P1152R3]

JF Bastien. [Deprecating volatile](https://wg21.link/p1152r3). 15 June 2019. URL: <https://wg21.link/p1152r3>

[P1152R4]

JF Bastien. [Deprecating volatile](https://wg21.link/P1152R4). 22 July 2019. URL: <https://wg21.link/P1152R4>